Intern Mini Project

Project: E-Commerce Order Analytics System

Duration: 3-4 weeks
Skills Tested: Python, SQL, Problem Solving

Background

You are joining a company that processes online orders. The raw data comes from multiple sources and is messy. Your job is to clean it, transform it, and generate business reports.

This project has following **phase**:

- **Phase:** Build everything using Python and SQL (local environment)

The Data

You will create 4 CSV files with sample data (at least 500 rows each):

1. orders.csv

```
order_id, customer_id, order_date, status, region_code
```

- status can be: PLACED, SHIPPED, DELIVERED, CANCELLED, RETURNED
- order_date format: YYYY-MM-DD HH:MM:SS
- Some rows have missing customer_id (marked as NULL or empty)

2. order_items.csv

```
item_id, order_id, product_id, quantity, unit_price, discount_percent
```

- discount_percent is between 0 and 100
- Some rows have negative quantity (these are returns)

3. products.csv

```
product_id, product_name, category, subcategory, cost_price
```

- category examples: Electronics, Clothing, Home, Books

4. customers.csv

```
customer_id, customer_name, email, registration_date, customer_type
```

- customer_type: REGULAR, PREMIUM, VIP

PHASE 1: Python & SQL (Local Environment)

Part 1: Data Generation (Python)

Write a Python script to generate the above 4 CSV files with realistic fake data.

Requirements:

- Data should have intentional issues:
  - 5% of orders should have NULL customer_id
  - 3% of order_items should have negative quantity
  - Some orders should have order_date in wrong format (DD-MM-YYYY)
  - Some product names should have extra spaces or mixed case
  - 2% of emails should be invalid (missing @ or domain)

Think about: How will you ensure order_id in order_items actually exists in orders table?

Part 2: Data Cleaning (Python)

Write Python functions (pandas allowed) to:

1. **clean_orders()** - Fix date formats, handle NULL customer_ids
2. **clean_products()** - Normalize product names (trim spaces, title case)

3. **validate_emails()** - Return list of customer_ids with invalid emails
4. **check_referential_integrity()** - Find order_items that reference non-existent orders

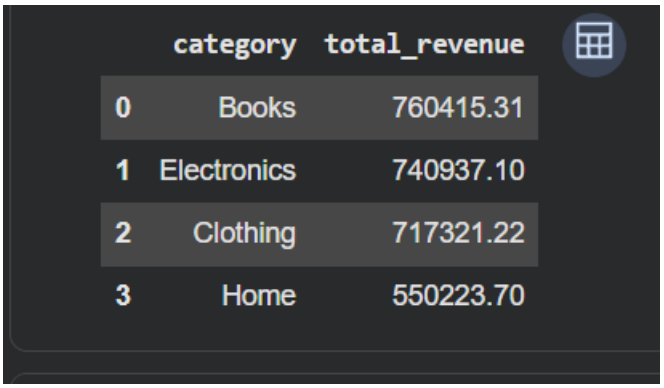
Output: Cleaned CSV files + a report of all issues found

Part 3: SQL Analysis

Using SQLite (or any SQL database), load your cleaned data and write queries for:

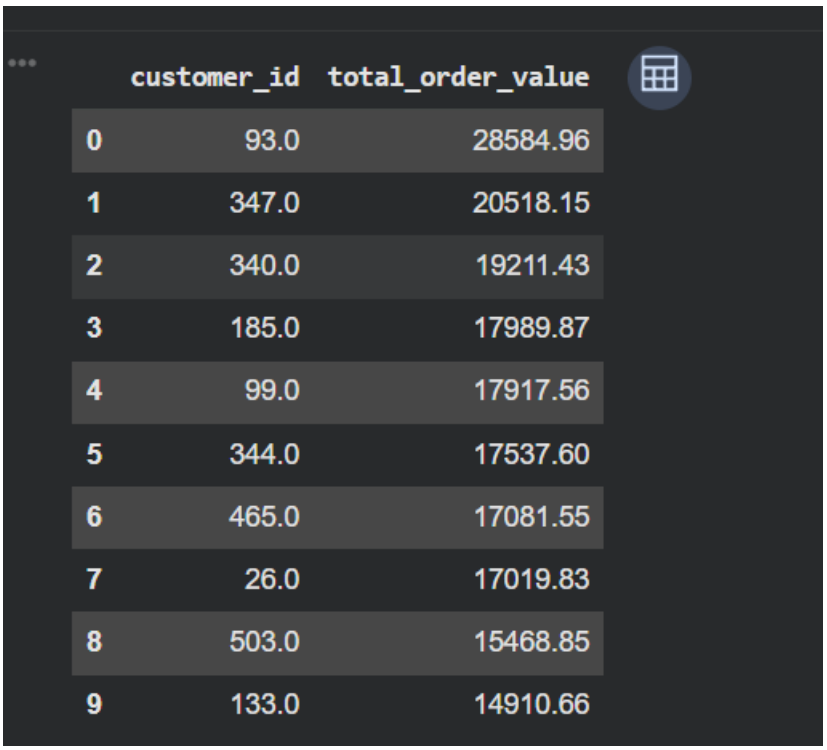
Basic Queries

1. Total revenue per category (revenue = quantity × unit_price × (1 - discount_percent/100))



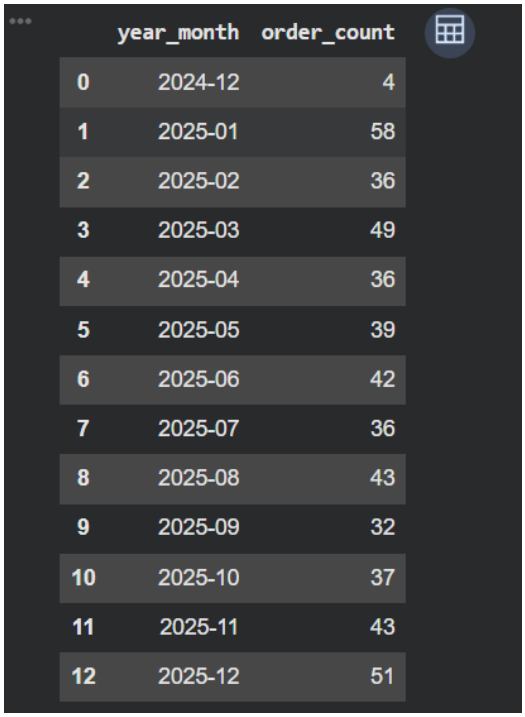
	category	total_revenue
0	Books	760415.31
1	Electronics	740937.10
2	Clothing	717321.22
3	Home	550223.70

2. Top 10 customers by total order value



	customer_id	total_order_value
0	93.0	28584.96
1	347.0	20518.15
2	340.0	19211.43
3	185.0	17989.87
4	99.0	17917.56
5	344.0	17537.60
6	465.0	17081.55
7	26.0	17019.83
8	503.0	15468.85
9	133.0	14910.66

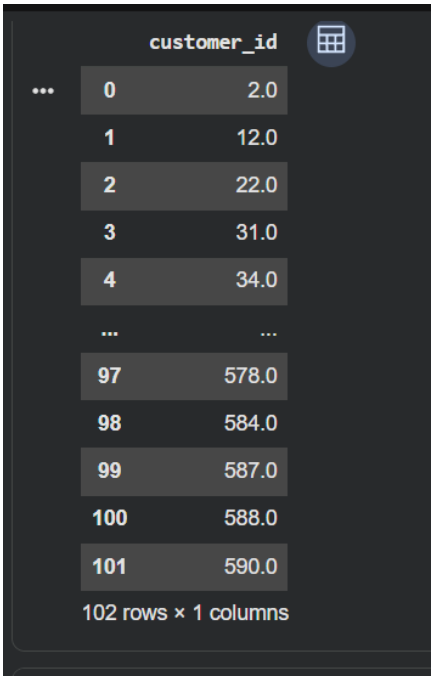
3. Month-wise order count for the last 12 months



	year_month	order_count
0	2024-12	4
1	2025-01	58
2	2025-02	36
3	2025-03	49
4	2025-04	36
5	2025-05	39
6	2025-06	42
7	2025-07	36
8	2025-08	43
9	2025-09	32
10	2025-10	37
11	2025-11	43
12	2025-12	51

Intermediate Queries

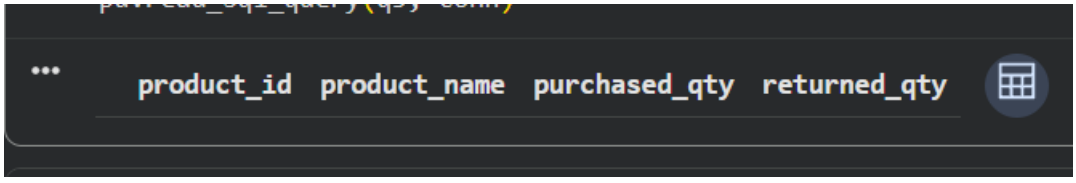
4. Find customers who placed orders but never had any item delivered



	customer_id
0	2.0
1	12.0
2	22.0
3	31.0
4	34.0
...	...
97	578.0
98	584.0
99	587.0
100	588.0
101	590.0

102 rows × 1 columns

5. Products that were ordered but had more returns than purchases



product_id	product_name	purchased_qty	returned_qty

6. Calculate the return rate (returned items / total items) per category

	category	returned_items	total_items	return_rate
0	Books	123	3445	0.0357
1	Home	85	2400	0.0354
2	Electronics	101	3315	0.0305
3	Clothing	56	3045	0.0184

Advanced Queries (Window Functions, CTEs, Subqueries)

7. **Running Totals with Window Functions**

Calculate running total of revenue per region, ordered by date.
Show: region_code, order_date, daily_revenue, running_total

	region_code	order_date	daily_revenue	running_total
0	APAC	2023-01-06	1292.59	1292.59
1	APAC	2023-01-14	1021.29	2313.88
2	APAC	2023-01-16	574.35	2888.23
3	APAC	2023-01-17	2088.52	4976.74
4	APAC	2023-01-18	796.17	5772.92
...	...	...	...	...
1189	NA	2025-12-17	235.00	558557.18
1190	NA	2025-12-18	-189.68	558367.50
1191	NA	2025-12-19	3331.50	561699.00
1192	NA	2025-12-25	1164.21	562863.22
1193	NA	2025-12-29	902.16	563765.38

1194 rows × 4 columns

8. **Ranking with DENSE_RANK**

For each category, rank products by total revenue.

Show: category, product_name, total_revenue, rank_in_category

Products with same revenue should have same rank.

	category	product_name	total_revenue	rank_in_category
0	Books	Bed Sheet Set	52098.28	1
1	Books	Coffee Mug Lite	48229.09	2
2	Books	Wall Clock Lite	39882.54	3
3	Books	Yoga Mat Plus	38440.74	4
4	Books	Winter Scarf Max	35590.00	5
...	...	...	...	...
125	Home	Backpack Plus	15950.29	24
126	Home	Bluetooth Speaker Max	14528.00	25
127	Home	Cotton T-Shirt Pro	14318.71	26
128	Home	Desk Organizer Lite	13407.73	27
129	Home	Yoga Mat Plus	12238.88	28

130 rows × 4 columns

9. LAG/LEAD Analysis

For each customer, calculate days between consecutive orders.

Show: customer_id, order_date, previous_order_date, days_gap

Flag customers with average gap > 30 days as "At Risk"

	customer_id	order_date	previous_order_date	days_gap	risk_flag
0	1.0	2023-02-23 00:00:00	None	NaN	At Risk
1	1.0	2023-08-16 22:19:03	2023-02-23 00:00:00	174.9	At Risk
2	1.0	2023-09-02 13:57:58	2023-08-16 22:19:03	16.7	At Risk
3	2.0	2023-05-31 11:36:00	None	NaN	At Risk
4	2.0	2024-04-27 19:25:24	2023-05-31 11:36:00	332.3	At Risk
...	...	...	...	...	...
1303	600.0	2024-01-30 00:06:48	2024-01-12 21:53:20	17.1	At Risk
1304	600.0	2024-04-17 02:44:57	2024-01-30 00:06:48	78.1	At Risk
1305	600.0	2024-11-28 08:38:10	2024-04-17 02:44:57	225.2	At Risk
1306	600.0	2025-01-20 05:00:30	2024-11-28 08:38:10	52.8	At Risk
1307	600.0	2025-03-03 18:25:21	2025-01-20 05:00:30	42.6	At Risk

1308 rows × 5 columns

10. CTE with Multiple Levels

Using CTEs, find:

- First, calculate monthly revenue per customer
- Then, categorize customers: 'High' (>10000), 'Medium' (5000-10000), 'Low' (<5000)
- Finally, show count of customers in each category per month

	year_month	tier	customer_count
0	2023-01	Low	34
1	2023-01	Medium	6
2	2023-02	Low	28
3	2023-02	Medium	4
4	2023-03	Low	26
...	...	...	...
64	2025-10	Medium	2
65	2025-11	Low	36
66	2025-11	Medium	2
67	2025-12	Low	38
68	2025-12	Medium	5

69 rows × 3 columns

11. NTILE for Segmentation

Divide customers into 4 quartiles based on total lifetime value.

Show: customer_id, total_value, quartile, quartile_label (Platinum/Gold/Silver/Bronze)

	customer_id	total_value	quartile	quartile_label
0	93.0	28584.96	1	Platinum
1	347.0	20518.15	1	Platinum
2	340.0	19211.43	1	Platinum
3	185.0	17989.87	1	Platinum
4	99.0	17917.56	1	Platinum
...	...	...	...	...
520	453.0	-427.84	4	Bronze
521	117.0	-1207.01	4	Bronze
522	363.0	-1411.12	4	Bronze
523	212.0	-1880.40	4	Bronze
524	341.0	-2670.95	4	Bronze

525 rows × 4 columns

12. Year-over-Year Comparison

Compare each month's revenue with same month previous year.
Show: year, month, revenue, prev_year_revenue, yoy_growth_percent
Handle cases where previous year data doesn't exist.

index	year	month	revenue	prev_year_revenue	yoy_growth_pe
0	2023	1	102587.67	NaN	NaN
1	2023	2	78948.53	NaN	NaN
2	2023	3	63587.81	NaN	NaN
3	2023	4	55716.81	NaN	NaN
4	2023	5	46622.66	NaN	NaN
5	2023	6	89525.87	NaN	NaN
6	2023	7	63201.62	NaN	NaN
7	2023	8	100192.45	NaN	NaN
8	2023	9	85844.08	NaN	NaN
9	2023	10	104046.69	NaN	NaN
10	2023	11	75925.6	NaN	NaN
11	2023	12	66643.54	NaN	NaN
12	2024	1	61929.36	102587.67	-39.63
13	2024	2	55197.38	78948.53	-30.08
14	2024	3	94590.24	63587.81	48.76
15	2024	4	61031.15	55716.81	9.54
16	2024	5	72610.82	46622.66	55.74
17	2024	6	50260.77	89525.87	-43.86
18	2024	7	69563.36	63201.62	10.07
19	2024	8	93830.17	100192.45	-6.35
20	2024	9	80305.12	85844.08	-6.45
21	2024	10	76585.85	104046.69	-26.39
22	2024	11	88131.81	75925.6	16.08
23	2024	12	98285.92	66643.54	47.48
24	2025	1	93897.78	61929.36	51.62

13. First/Last Value Analysis

For each customer, show their first purchased category and most recent purchased category.
Flag if they are different (category_shift = 'Yes'/'No')

	customer_id	first_category	last_category	category_shift
0	1.0	Home	Electronics	Yes
1	2.0	Home	Electronics	Yes
2	3.0	Electronics	Clothing	Yes
3	5.0	Electronics	Clothing	Yes
4	6.0	Clothing	Electronics	Yes
...	...	...	...	...
520	596.0	Books	Books	No
521	597.0	Clothing	Clothing	No
522	598.0	Clothing	Home	Yes
523	599.0	Electronics	Electronics	No
524	600.0	Books	Clothing	Yes

525 rows × 4 columns

14. **Cumulative Distribution**

Calculate what percentage of total revenue comes from top N% of customers.

Show: customer_id, revenue, cumulative_revenue, cumulative_percent

	customer_id	revenue	cumulative_revenue	cumulative_percent
0	93.0	28584.96	28584.96	1.09
1	347.0	20518.15	49103.11	1.87
2	340.0	19211.43	68314.55	2.60
3	185.0	17989.87	86304.42	3.28
4	99.0	17917.56	104221.97	3.97
...	...	...	...	...
520	453.0	-427.84	2635171.87	100.27
521	117.0	-1207.01	2633964.86	100.23
522	363.0	-1411.12	2632553.74	100.17
523	212.0	-1880.40	2630673.34	100.10
524	341.0	-2670.95	2628002.40	100.00

525 rows × 4 columns

15. **Complex CTE: Cohort Analysis**

Group customers by their registration month (cohort).

For each cohort, calculate:

- How many ordered in month 0 (registration month)
- How many ordered in month 1, month 2, month 3
- Retention rate for each month

	cohort_month	month_offset	active_customers	cohort_size	retention_rate
0	2023-01	2	1	12	8.33
1	2023-01	3	2	12	16.67
2	2023-02	1	1	16	6.25
3	2023-02	3	1	16	6.25
4	2023-03	0	2	18	11.11
...	...	...	...	...	...
82	2025-08	2	2	19	10.53
83	2025-09	3	2	16	12.50
84	2025-10	1	1	16	6.25
85	2025-10	2	1	16	6.25
86	2025-11	1	2	24	8.33

87 rows × 5 columns

16. **Self-Join with Window Function**

Find products frequently bought together.

Show: product_a, product_b, times_bought_together

Exclude same product pairs and duplicates (A-B and B-A should appear once)

	product_a	product_b	times_bought_together
0	Running Shoes Pro	Denim Jacket Lite	9
1	Wireless Headphones Plus	Smart Watch Pro	8
2	Backpack Max	Bluetooth Speaker Max	7
3	Cotton T-Shirt Lite	Wireless Headphones Plus	7
4	Wall Clock Max	Running Shoes Pro	7
5	Winter Scarf Max	Bluetooth Speaker Max	7
6	Backpack Max	Cotton T-Shirt Pro	6
7	Backpack Max	Denim Jacket Lite	6
8	Coffee Mug Lite	Backpack Max	6
9	Coffee Mug Lite	Cotton T-Shirt Plus	6
10	Coffee Mug Pro	Backpack Lite	6
11	Comic Bundle Plus	Phone Case Pro	6
12	Cookbook	Coffee Mug Lite	6
13	Cotton T-Shirt	Coffee Mug Lite	6
14	Desk Organizer Plus	Smart Watch Pro	6
15	Novel Lite	Smart Watch Pro	6
16	Phone Case Pro	Wireless Headphones Plus	6
17	Wall Clock Max	Phone Case Pro	6
18	Yoga Mat Max	Yoga Mat Plus	6
19	Backpack Max	Desk Organizer Plus	5

Part 4: Python + SQL Integration

Build a simple command-line tool that:

1. Takes user input for report type (daily/weekly/monthly)
2. Takes date range as input
3. Connects to SQLite database
4. Generates a summary report showing:
    - Total orders, revenue, unique customers
    - Top 3 products
    - Comparison with previous period (% change)

No external libraries except sqlite3

```
%%writefile report_cli.py

import sqlite3
import argparse
from datetime import datetime, timedelta

DB_PATH = "ecommerce.db"

REVENUE_EXPR = "oi.quantity * oi.unit_price * (1 - oi.discount_percent/100.0)"

def get_period_summary(conn, start_date, end_date):
    """Return dict with total_orders, revenue, unique_customers, top_3_products
    for orders with order_date in [start_date, end_date)."""
    cur = conn.cursor()

    cur.execute(f"""
        SELECT
            COUNT(DISTINCT o.order_id) AS total_orders,
            COALESCE(SUM({REVENUE_EXPR}), 0) AS revenue,
            COUNT(DISTINCT o.customer_id) AS unique_customers
        FROM orders o
        LEFT JOIN order_items oi ON oi.order_id = o.order_id
        WHERE o.order_date >= ? AND o.order_date < ?
    """, (start_date, end_date))
    total_orders, revenue, unique_customers = cur.fetchone()

    cur.execute(f"""
        SELECT p.product_name, SUM({REVENUE_EXPR}) AS product_revenue
        FROM order_items oi
        JOIN orders o ON o.order_id = oi.order_id
        JOIN products p ON p.product_id = oi.product_id
        WHERE o.order_date >= ? AND o.order_date < ?
        GROUP BY p.product_name
        ORDER BY product_revenue DESC
```

```
        LIMIT 3
    """, (start_date, end_date))
    top_products = cur.fetchall()

    return {
        "total_orders": total_orders or 0,
        "revenue": revenue or 0.0,
        "unique_customers": unique_customers or 0,
        "top_products": top_products,
    }

def previous_period(start_date, end_date):
    """Given [start, end), return the immediately preceding period of equal length."""
    fmt = "%Y-%m-%d"
    start = datetime.strptime(start_date, fmt)
    end = datetime.strptime(end_date, fmt)
    length = end - start
    prev_end = start
    prev_start = start - length
    return prev_start.strftime(fmt), prev_end.strftime(fmt)

def pct_change(current, previous):
    if previous in (0, None):
        return None
    return round(100.0 * (current - previous) / previous, 2)

def date_range_for_report_type(report_type, anchor_date=None):
    """Compute a default [start, end) window for daily/weekly/monthly, ending at anchor_date."""
    fmt = "%Y-%m-%d"
    end = datetime.strptime(anchor_date, fmt) if anchor_date else datetime.today()
    if report_type == "daily":
        start = end - timedelta(days=1)
    elif report_type == "weekly":
        start = end - timedelta(days=7)
    elif report_type == "monthly":
        start = end - timedelta(days=30)
    else:
        raise ValueError("report_type must be daily, weekly, or monthly")
    return start.strftime(fmt), end.strftime(fmt)

def print_report(report_type, start_date, end_date, conn):
    current = get_period_summary(conn, start_date, end_date)
    prev_start, prev_end = previous_period(start_date, end_date)
    previous = get_period_summary(conn, prev_start, prev_end)

    print("\n" + "=" * 50)
    print(f"{report_type.upper()} REPORT: {start_date} to {end_date}")
    print("=" * 50)
    print(f"Total Orders:      {current['total_orders']}"
          f"  ({pct_change(current['total_orders'], previous['total_orders'])}% vs previous period)")
    print(f"Revenue:           {current['revenue']:.2f}"
          f"  ({pct_change(current['revenue'], previous['revenue'])}% vs previous period)")
    print(f"Unique Customers:  {current['unique_customers']}"
          f"  ({pct_change(current['unique_customers'], previous['unique_customers'])}% vs previous period)")
    print("\nTop 3 Products:")
    if current["top_products"]:
        for name, rev in current["top_products"]:
            print(f"  - {name}: {rev:.2f}")
```

```
    else:
        print("  (no sales in this period)")
    print(f"\nPrevious period: {prev_start} to {prev_end}")
    print("=" * 50)

def main():
    parser = argparse.ArgumentParser(description="E-commerce summary report generator")
    parser.add_argument("--type", choices=["daily", "weekly", "monthly"], help="Report type")
    parser.add_argument("--start", help="Start date YYYY-MM-DD")
    parser.add_argument("--end", help="End date YYYY-MM-DD")
    parser.add_argument("--db", default=DB_PATH, help="Path to SQLite database")
    args = parser.parse_args()

    report_type = args.type
    start_date = args.start
    end_date = args.end

    # fall back to interactive prompts if not supplied on the command line
    if report_type is None:
        report_type = input("Report type (daily/weekly/monthly): ").strip().lower()
        while report_type not in ("daily", "weekly", "monthly"):
            report_type = input("Please enter daily, weekly, or monthly: ").strip().lower()

    if start_date is None or end_date is None:
        use_custom = input("Enter custom date range? (y/n): ").strip().lower()
        if use_custom == "y":
            start_date = input("Start date (YYYY-MM-DD): ").strip()
            end_date = input("End date (YYYY-MM-DD): ").strip()
        else:
            start_date, end_date = date_range_for_report_type(report_type)

    conn = sqlite3.connect(args.db)
    try:
        print_report(report_type, start_date, end_date, conn)
    finally:
        conn.close()

if __name__ == "__main__":
    main()
```

Part 5: Edge Case Handling

Write test cases (as Python functions) that verify:

1. What happens when order_items has an order_id not in orders?
2. What happens when discount_percent > 100?
3. What happens when quantity is 0?
4. What happens when order_date is in the future?

```
%%writefile test_edge_cases.py

from datetime import datetime, timedelta
import pandas as pd

from clean_data import check_referential_integrity

def test_order_item_with_missing_order():
    """order_items row pointing at a non-existent order_id should be flagged
    by check_referential_integrity, not silently dropped or crash the pipeline."""
    orders = pd.DataFrame({
        "order_id": [1, 2, 3],
        "customer_id": [10, 11, 12],
    })
    order_items = pd.DataFrame({
        "item_id": [100, 101],
        "order_id": [1, 999],
        "product_id": [5, 6],
        "quantity": [2, 1],
        "unit_price": [10.0, 20.0],
        "discount_percent": [0, 0],
    })

    bad_rows = check_referential_integrity(order_items, orders)

    assert len(bad_rows) == 1, f"expected 1 orphaned row, got {len(bad_rows)}"
    assert bad_rows.iloc[0]["order_id"] == 999
    print("PASS: test_order_item_with_missing_order")

def test_discount_percent_over_100():
    """discount_percent > 100 is invalid input (would produce negative revenue).
    We verify it's detectable so the pipeline can flag/reject it rather than
    silently generating nonsense revenue numbers."""
    order_items = pd.DataFrame({
        "item_id": [1, 2],
        "order_id": [1, 1],
        "product_id": [1, 2],
        "quantity": [1, 1],
        "unit_price": [100.0, 50.0],
        "discount_percent": [150, 20],
    })

    invalid = order_items[order_items["discount_percent"] > 100]
    assert len(invalid) == 1
    assert invalid.iloc[0]["item_id"] == 1


    revenue = order_items["quantity"] * order_items["unit_price"] * (1 - order_items["discount_percent"] / 100)
    assert revenue.iloc[0] < 0, "an unflagged discount_percent > 100 produces negative revenue"
    print("PASS: test_discount_percent_over_100 (correctly detects invalid rows)")

def test_quantity_zero():
    """quantity == 0 is a valid-looking but meaningless line item: it contributes
    zero revenue and shouldn't be counted as either a purchase or a return."""
    order_items = pd.DataFrame({
        "item_id": [1, 2, 3],
        "order_id": [1, 1, 1],
        "product_id": [1, 2, 3],
        "quantity": [0, 5, -2],
        "unit_price": [10.0, 10.0, 10.0],
        "discount_percent": [0, 0, 0],
    })
```

```
    revenue = order_items["quantity"] * order_items["unit_price"] * (1 - order_items["discount_percent"] / 100)
    assert revenue.iloc[0] == 0, "zero quantity should contribute exactly zero revenue"

    zero_qty_rows = order_items[order_items["quantity"] == 0]
    assert len(zero_qty_rows) == 1
    print("PASS: test_quantity_zero")

def test_future_order_date():
    """An order_date in the future is a data-quality problem (clock skew, bad
    manual entry, etc.). It should be detectable rather than silently included
    in 'current' reporting periods."""
    future_date = datetime.now() + timedelta(days=30)
    orders = pd.DataFrame({
        "order_id": [1, 2],
        "customer_id": [1, 2],
        "order_date": [datetime.now() - timedelta(days=1), future_date],
    })

    future_orders = orders[orders["order_date"] > datetime.now()]
    assert len(future_orders) == 1
    assert future_orders.iloc[0]["order_id"] == 2
    print("PASS: test_future_order_date")

def run_all():
    tests = [
        test_order_item_with_missing_order,
        test_discount_percent_over_100,
        test_quantity_zero,
        test_future_order_date,
    ]
    failures = 0
    for t in tests:
        try:
            t()
        except AssertionError as e:
            failures += 1
            print(f"FAIL: {t.__name__} -> {e}")
    print(f"\n{len(tests) - failures}/{len(tests)} tests passed")
    if failures:
        raise SystemExit(1)

if __name__ == "__main__":
    run_all()
```
